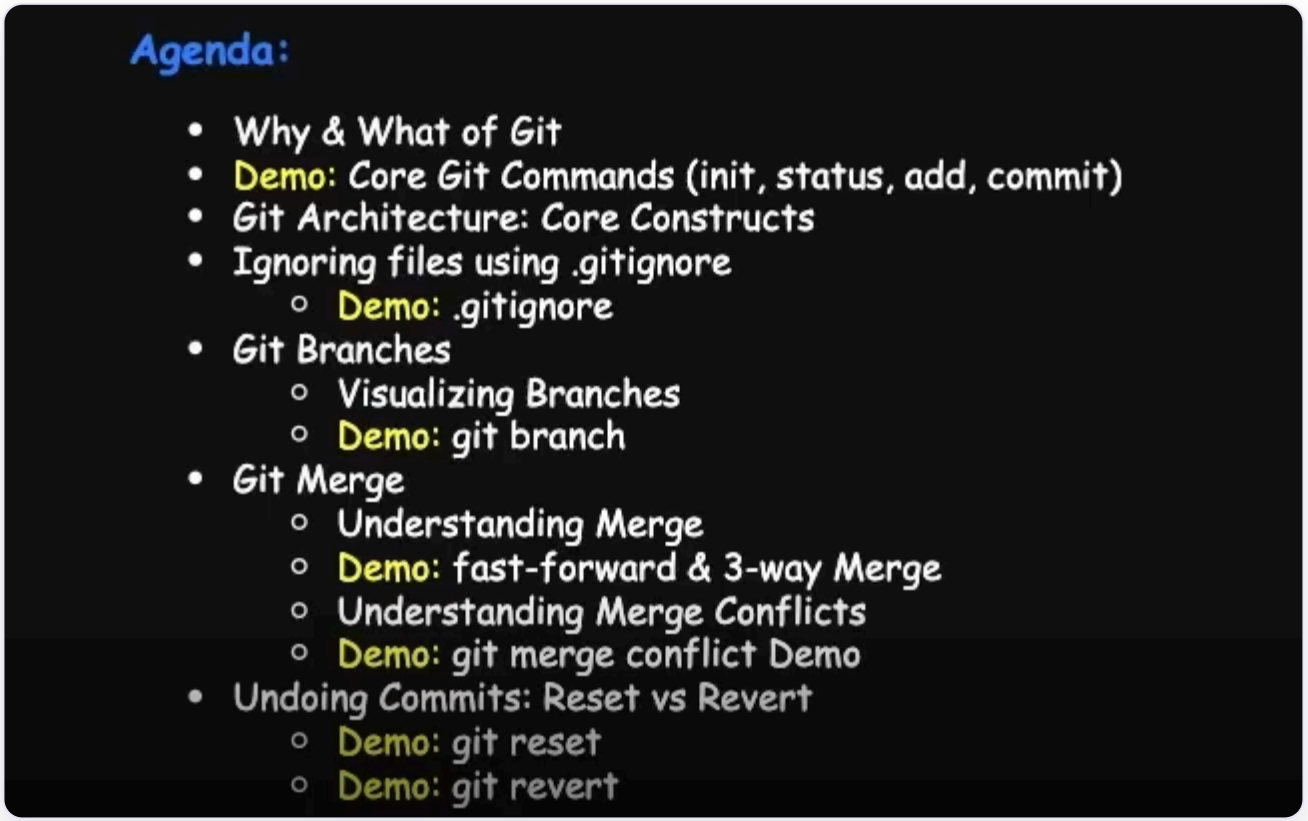
GIT TRACK · SCREENSHOTS

Chapter 01 — Git Fundamentals init, Architecture, Commits, config & .gitignore

Git Masterclass · Part 1 · stills from the session, in order · 2026-10-13

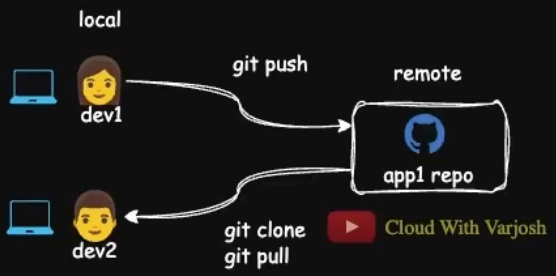
39 screenshots


Agenda:
Why & What of Git
Demo: Core Git Commands (init, status, add, commit)
Git Architecture: Core Constructs
Ignoring files using .gitignore
Demo: .gitignore
Git Branches
Visualizing Branches
Demo: git branch
Git Merge
Understanding Merge
Demo: fast-forward & 3-way Merge
Understanding Merge Conflicts
Demo: git merge conflict Demo
Undoing Commits: Reset vs Revert
Demo: git reset
Demo: git revert

Git Fundamentals for Modern Engineering

Why Git exists?

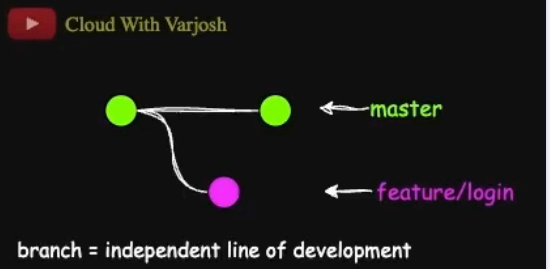
- Teams frequently overwrote each other's work
- No reliable history of code changes
- Rolling back broken changes was difficult
- Collaboration across engineers became difficult

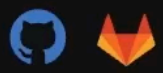


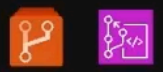
What is Git? (Linus Torvalds in 2005)

- Distributed version control system (DVCS) for tracking changes in application code & infra.
- Every developer has full repository history locally
- Commits create snapshots of project changes
- Push (local → remote) and pull (remote → local) synchronize repositories
- Branches enable isolated parallel development
- git vs GitHub
- Why Distributed?

Cloud With Varjosh







Summary:
Change → Git solves it
History → commits
Isolation → branches
Collaboration → remotes

Where Git Repositories Are Hosted

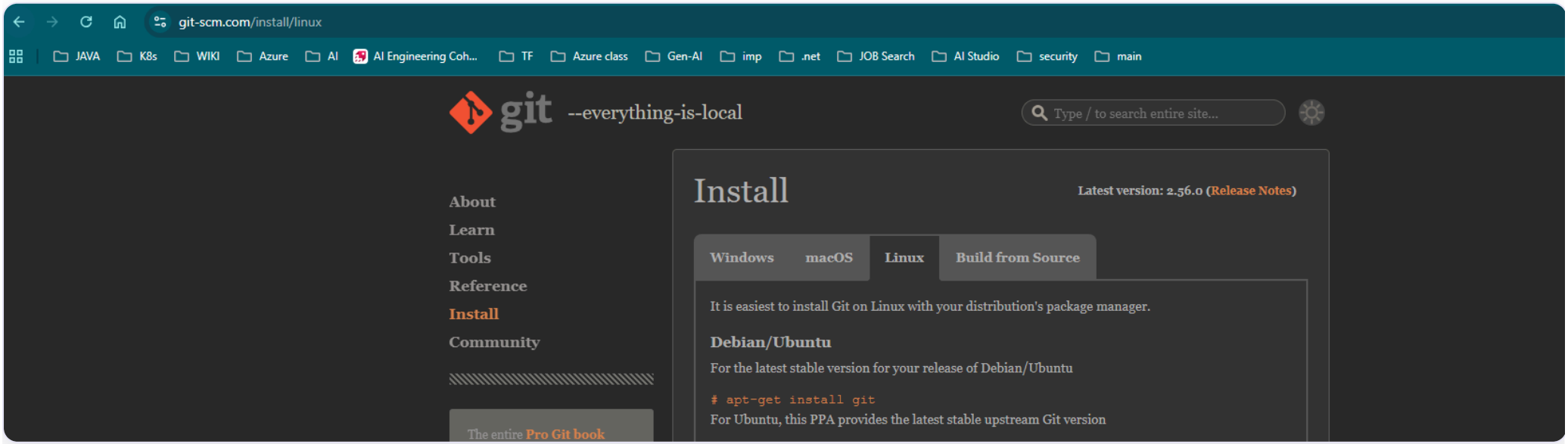
- Cloud Platforms: GitHub, GitLab, Bitbucket
- Cloud Provider suites: Azure Repos, AWS CodeCommit
- Self-hosted: GitLab CE/EE, Gitea, Gerrit

Remote repositories enable collaboration, pull requests, CI/CD triggers, centralized backup, and access control.

Git in Modern DevOps

- Git stores application & infrastructure definitions
- Kubernetes, Terraform, Ansible configurations versioned
- GitOps uses Git as desired system state

03 / 39 git-scm.com install page



04 / 39 git -v → 2.43.0; apt install git

```
root@cka-w2:~# git -v
git version 2.43.0
root@cka-w2:~# apt install git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
```

05 / 39 git status in a normal folder → fatal: not a git repository

```
root@cka-w2:~#
root@cka-w2:~# mkdir project-files
root@cka-w2:~# cd project-files/
root@cka-w2:~/project-files# ls
root@cka-w2:~/project-files# git status
fatal: not a git repository (or any of the parent directories): .git
root@cka-w2:~/project-files#
```

git init → Initialized empty Git repository (master hint)

```
root@cka-w2:~/project-files# git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /root/project-files/.git/
root@cka-w2:~/project-files#
```

07 / 39 git branch --show-current → master

```
root@cka-w2:~/project-files# git branch --show-current
master
root@cka-w2:~/project-files#
```

ls -ltrha → hidden .git folder

```
root@cka-w2 ~/project-files
$ ls -ltrha
total 12K
drwxr-xr-x 3 root root 4.0K Oct 13 06:15 .
drwx------ 9 root root 4.0K Oct 13 06:22 ..
drwxr-xr-x 7 root root 4.0K Oct 13 06:25 .git
root@cka-w2 ~/project-files
$
```

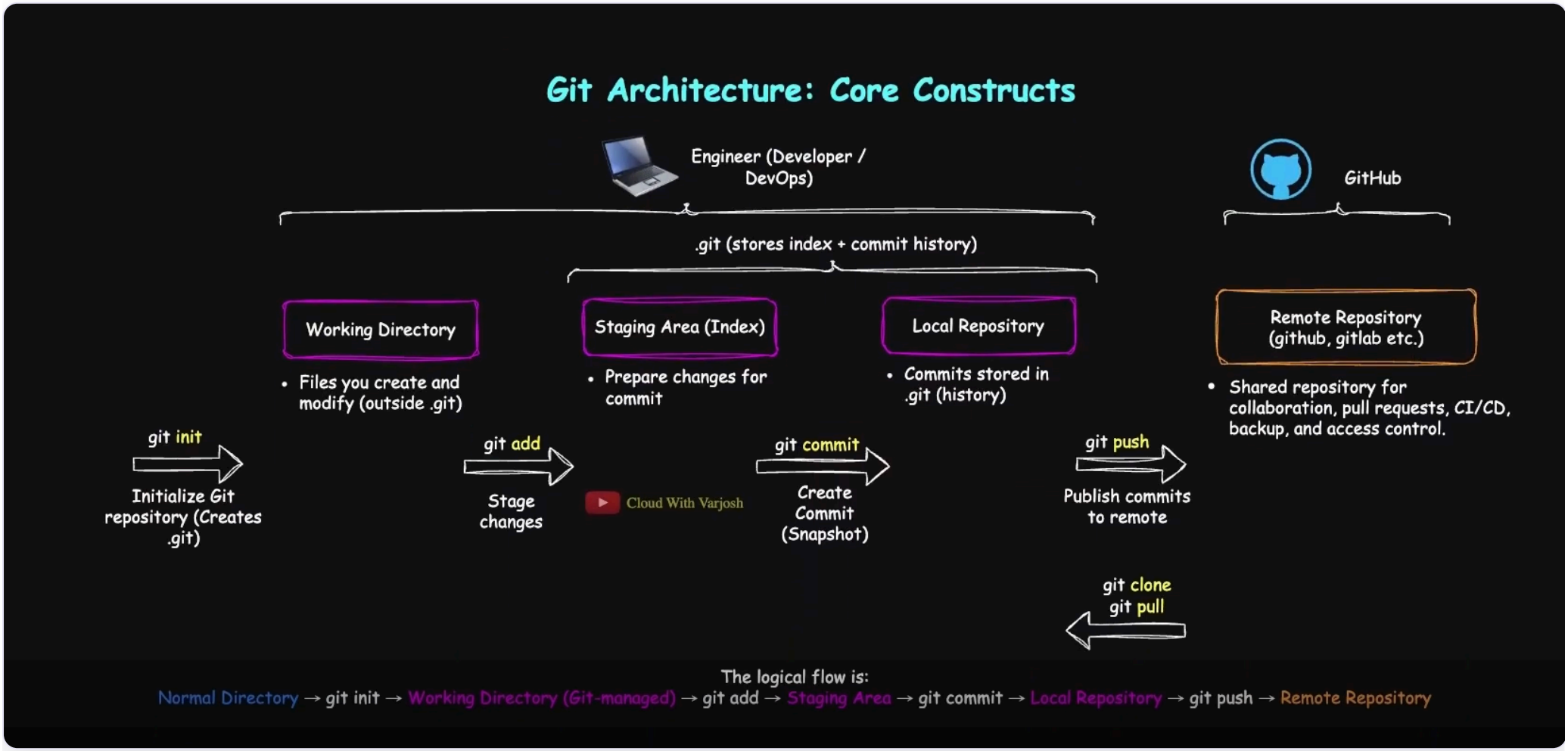
Inside .git — 10 directories, 18 files

```
├── branches
├── config
├── description
├── HEAD
├── hooks
│   ├── applypatch-msg.sample
│   ├── commit-msg.sample
│   ├── fsmonitor-watchman.sample
│   ├── post-update.sample
│   ├── pre-applypatch.sample
│   ├── pre-commit.sample
│   ├── pre-merge-commit.sample
│   ├── prepare-commit-msg.sample
│   ├── pre-push.sample
│   ├── pre-rebase.sample
│   ├── pre-receive.sample
│   ├── push-to-checkout.sample
│   ├── sendemail-validate.sample
│   └── update.sample
├── info
│   └── exclude
├── objects
│   ├── info
│   └── pack
└── refs
    ├── heads
    └── tags

10 directories, 18 files
```

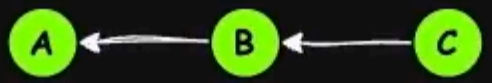
tree .git

```
root@cka-w2 ~/project-files
$ tree .git
.git
```


Git Architecture: Core Constructs
Engineer (Developer / DevOps)
GitHub
.git (stores index + commit history)
Working Directory
Staging Area (Index)
Local Repository
Remote Repository (github, gitlab etc.)
• Files you create and modify (outside .git)
• Prepare changes for commit
• Commits stored in .git (history)
• Shared repository for collaboration, pull requests, CI/CD, backup, and access control.
git init
Initialize Git repository (Creates .git)
git add
Stage changes
Cloud With Varjosh
git commit
Create Commit (Snapshot)
git push
Publish commits to remote
git clone
git pull
The logical flow is:
Normal Directory → git init → Working Directory (Git-managed) → git add → Staging Area → git commit → Local Repository → git push → Remote Repository

What is a Commit?

- Immutable snapshot of the project at a point in time
- Captures the exact state of files at that moment
- Stores metadata: author, timestamp, message, parent commit
- References a parent commit, to form the commit history



C parent → B
B parent → A
A parent → none

When we say a commit is immutable, we mean that the commit object and everything it represents (including file contents, directory structure, and metadata) cannot be changed once created.

However, the files and directories themselves can be modified in the working directory, and those changes can be recorded as a new commit.

Git does not modify an existing commit. Instead, it creates a new commit with a new hash. More on this later.

git log on an empty repo

```
root@cka-w2 ~/project-files
$ git log
fatal: your current branch 'master' does not have any commits yet
root@cka-w2 ~/project-files
$
```

git status — No commits yet

```
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
root@cka-w2 ~/project-files
$
```

 touch file1.txt → untracked

```
root@cka-w2 ~/project-files
$ touch file1.txt
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file1.txt

nothing added to commit but untracked files present (use "git add" to track)
root@cka-w2 ~/project-files
```

git add file1.txt → Changes to be committed

```
root@cka-w2 ~/project-files
$ git add file1.txt
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt

root@cka-w2 ~/project-files
$
```

git rm --cached → back to untracked

```
root@cka-w2 ~/project-files
$ git rm --cached file1.txt
rm 'file1.txt'
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file1.txt

nothing added to commit but untracked files present (use "git add" to track)
root@cka-w2 ~/project-files
$
```

19 / 39 Staged file1.txt is empty

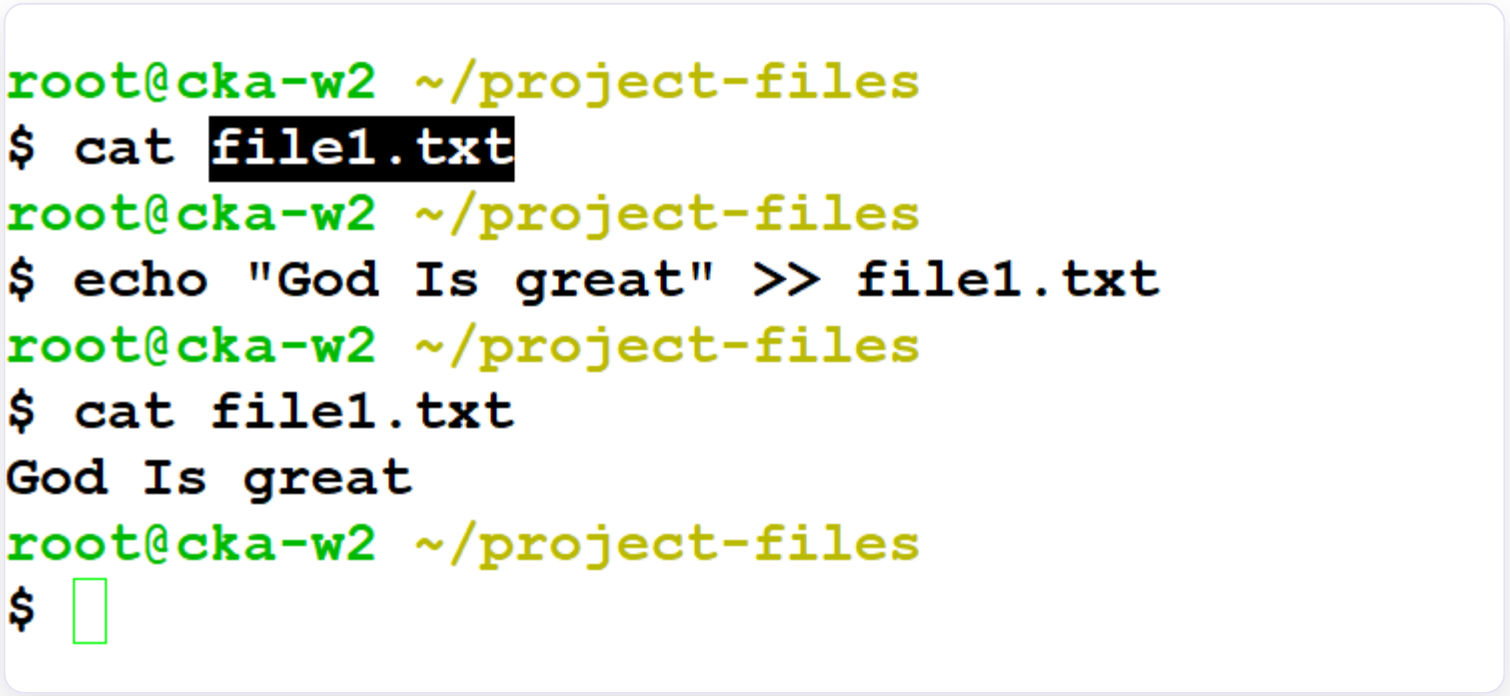
```
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt

root@cka-w2 ~/project-files
$ cat file1.txt
root@cka-w2 ~/project-files
$
```

echo “God Is great” >> file1.txt



21 / 39 Staged AND modified at the same time

```
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt

root@cka-w2 ~/project-files
$
```

git restore file1.txt → edit discarded

```
root@cka-w2 ~/project-files
$ cat file1.txt
God Is great
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt

root@cka-w2 ~/project-files
$ git restore file1.txt
root@cka-w2 ~/project-files
$ cat file1.txt
root@cka-w2 ~/project-files
$
```

Edit again + git add

```
root@cka-w2 ~/project-files
$ echo "God Is great" >> file1.txt
root@cka-w2 ~/project-files
$ cat file1.txt
God Is great
root@cka-w2 ~/project-files
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   file1.txt

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt

root@cka-w2 ~/project-files
$ git add file1.txt
root@cka-w2 ~/project-files
$
```

Commit fails — Author identity unknown

```
root@cka-w2 ~/project-files
$ git commit -m "inital commit: add file1.txt" .
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'root@cka-w2.(none)')
root@cka-w2 ~/project-files
$
```

git config --global user.name

```
$ git config --global user.name vishal
root@cka-w2 ~/project-files
```

git config --global user.email

```
$ git config --global user.email vishalbhilare@test.com
root@cka-w2 ~/project-files
```

27 / 39 First commit — root-commit 255d82f

```
$ git commit -m "inital commit: add file1.txt" .
[master (root-commit) 255d82f] inital commit: add file1.txt
 1 file changed, 1 insertion(+)
 create mode 100644 file1.txt
root@cka-w2 ~/project-files (master)
$
```

git log — hash, author, date, message

```
root@cka-w2 ~/project-files (master)
$ git log
commit 255d82feada282ef095ccab1a9f1aec1b30d3273 (HEAD -> master)
Author: vishal <vishalbhilare@test.com>
Date:   Tue Oct 13 07:49:28 2026 +0000

    inital commit: add file1.txt
root@cka-w2 ~/project-files (master)
$
```

```
<type>: <short description>
```

Recommended length

- The **short description** should ideally be **≤ 50 characters**.
- It should describe the change using the **imperative mood** (e.g., *add*, *fix*, *update*, not *added* or *fixed*).

Common commit types

- `feat:` → new feature
- `fix:` → bug fix
- `docs:` → documentation changes
- `style:` → formatting or whitespace changes (no logic change)
- `refactor:` → restructuring code without changing behavior
- `test:` → adding or modifying tests
- `chore:` → maintenance tasks (dependencies, tooling, build updates)

Examples

```
feat: add ArgoCD application deployment pipeline
fix: resolve failing Terraform state lock issue
chore: upgrade Kubernetes cluster to v1.29
feat: implement password strength validation
fix: prevent null pointer in user authentication service
```

Where Are Git Configuration Values Stored?

Every time you run `git config`, Git writes the configuration to a file. The storage location depends on the scope flag used.

Configuration Scope Flags

- `--system` → Applies machine-wide for all users. Stored in: `/etc/gitconfig` (or equivalent system-level config file).
- `--global` → Applies to the current OS user across all repositories. Stored in: `~/.gitconfig` or `~/.config/git/config`.
- `--local` → Applies only to the current repository. Stored in: `.git/config` inside that repository.
- `--worktree` → Applies to a specific worktree (in multi-worktree setups). Stored in: `.git/worktrees/<worktree-name>/config.worktree`.
- `--file <path>` → Writes configuration directly to the file you specify. Stored exactly at the custom path provided.

cat ~/.gitconfig

```
root@cka-w2 ~
$ cat .gitconfig
[user]
        name = vishal
        email = vishalbhilare@test.com
root@cka-w2 ~
$ pwd
/root
root@cka-w2 ~
$
```

file2.txt commit + git log

```
root@cka-w2 ~/project-files (master)
$ touch file2.txt
root@cka-w2 ~/project-files (master)
$ git add file2.txt
root@cka-w2 ~/project-files (master)
$ git commit -m "Addition : Add file2.txt"
[master 930b564] Addition : Add file2.txt
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file2.txt
root@cka-w2 ~/project-files (master)
$ git log
commit 930b564d382bfd03641a7af98da97771453dbd5e (HEAD -> master)
Author: vishal <vishalbhilare@test.com>
Date:   Tue Oct 13 07:59:58 2026 +0000

    Addition : Add file2.txt

commit 255d82feada282ef095ccab1a9f1aec1b30d3273
Author: vishal <vishalbhilare@test.com>
Date:   Tue Oct 13 07:49:28 2026 +0000

    inital commit: add file1.txt
root@cka-w2 ~/project-files (master)
$
```

```
$ git log --oneline
930b564 (HEAD -> master) Addition : Add file2.txt
255d82f inital commit: add file1.txt
root@cka-w2 ~/project-files (master)
$
```

Ignoring Files with `.gitignore`

During development, operating systems, programming languages, and tooling often generate files that **should not be tracked or included in version history**. These typically fall into two categories:

1. **Sensitive files (security-related)** Files that contain secrets or credentials, such as **API keys, tokens, private keys, and environment configuration files**.
2. **Generated or environment-specific files** Files automatically created by operating systems, build tools, or development environments, such as **log files, temporary files, build artifacts, and system-generated metadata**.

Tracking such files can lead to **security risks, unnecessary repository size, and noisy commit history**.

Git provides a mechanism to exclude such files from tracking using a `.gitignore` file.

The `.gitignore` file defines patterns for files and directories that Git should **exclude when determining which untracked files to show and stage**.

> **Important:** `.gitignore` only applies to **untracked files**. If a file is already tracked by Git, adding it to `.gitignore` will not stop Git from tracking it. To stop tracking such a file, it must first be removed from the index using:

```
git rm --cached <file>
```

my.key and my.log are untracked

```
root@cka-w2 ~/project-files (master)
$ touch my.log
root@cka-w2 ~/project-files (master)
$ touch my.key
root@cka-w2 ~/project-files (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        my.key
        my.log

nothing added to commit but untracked files present (use "git add" to track)
root@cka-w2 ~/project-files (master)
$
```

.gitignore — *.log, *.key

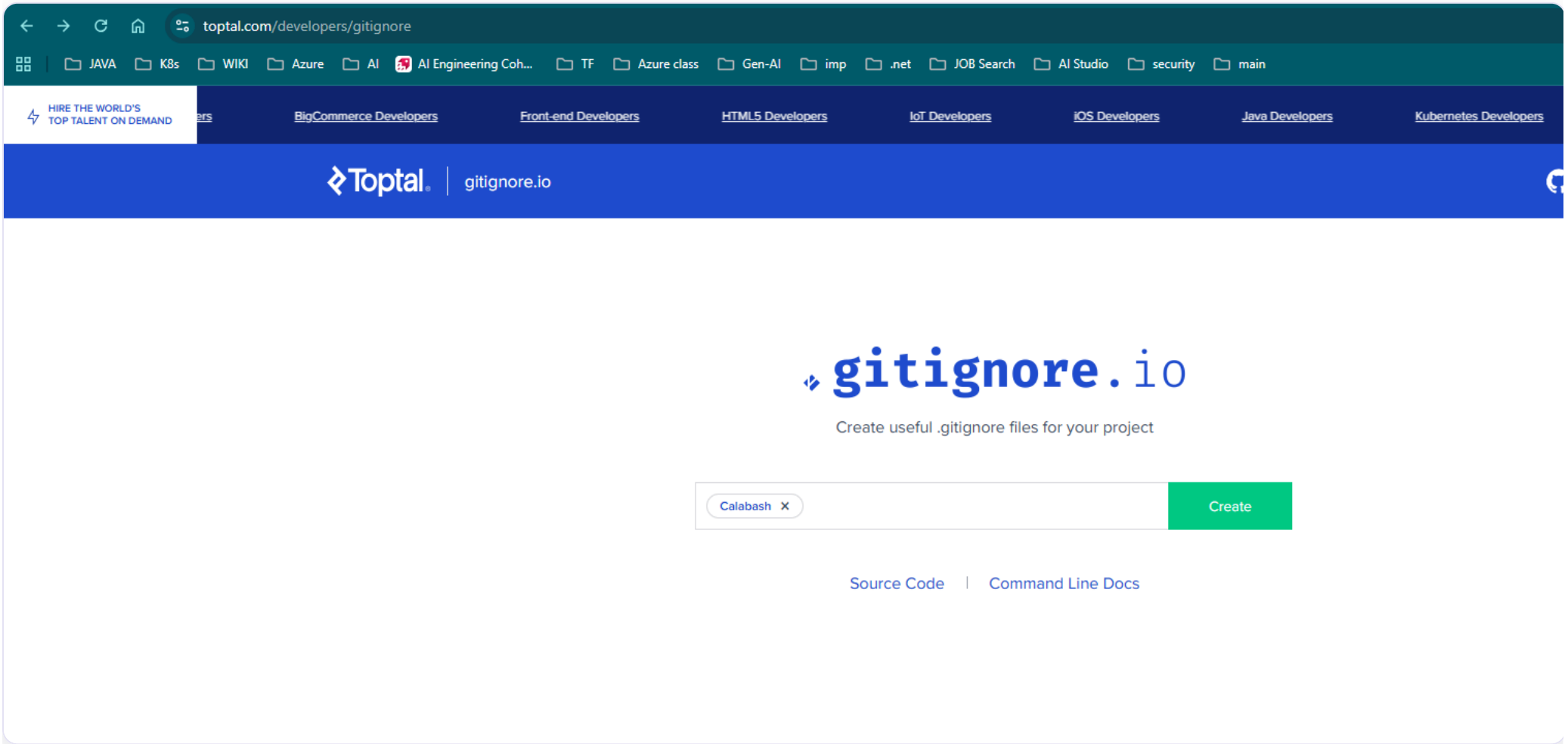
```
root@cka-w2 ~/project-files (master)
$ vim .gitignore
root@cka-w2 ~/project-files (master)
$ cat .gitignore
*.log
*.key
root@cka-w2 ~/project-files (master)
$
```

git status → only .gitignore remains

```
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)
root@cka-w2 ~/project-files (master)
$
```

gitignore.io generator (Toptal)



Generated .gitignore example

```
# Created by https://www.toptal.com/developers/gitignore/api/calabash
# Edit at https://www.toptal.com/developers/gitignore?templates=calabash

### Calabash ###
# Calabash / Cucumber
rerun/
reports/
screenshots/
screenshot*.png
test-servers/

# bundler
.bundle
vendor

# End of https://www.toptal.com/developers/gitignore/api/calabash
```